Università degli Studi di Palermo

dipartimento di ingegneria unipa

Introduzione a matplotlib

INTELLIGENZA ARTIFICIALE PER L'INGEGNERIA BIOMEDICA (EX ROBOTICA MEDICA)

a.a. 2025/2026

Prof. Roberto Pirrone

CHILAB

Importazione e struttura di base

matplotlib.pyplot — l'interfaccia di alto livello ispirata a MATLAB

Importazione

Python

```python
# Importazione convenzionale
import matplotlib.pyplot as plt
import numpy as np

# Backend non-interattivo (script, salva su file)
import matplotlib
matplotlib.use('Agg')         # nessuna finestra

# In Jupyter Notebook
%matplotlib inline            # grafici nel notebook
```

Stile pyplot: figura implicita

Python – stile pyplot

```python
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2*np.pi, 200)
plt.plot(x, np.sin(x), label='sin(x)')
plt.plot(x, np.cos(x), label='cos(x)')
plt.xlabel('x'); plt.ylabel('y')
plt.title('Funzioni trigonometriche')
plt.legend()
plt.tight_layout()
plt.savefig('trig.png', dpi=150)  # salva su file
plt.show()                         # apre finestra
plt.close()                        # libera memoria
```

Architettura a livelli

Livello scripting (pyplot)
plt.plot(), plt.show() — stato globale implicito

Livello Artist (OO API)
fig, ax = plt.subplots() — controllo esplicito

Livello Backend
rendering su schermo, file PNG / SVG / PDF

API orientata agli oggetti (consigliata)

Python – OO API (preferita)

```python
# fig: contenitore; ax: sistema di assi
fig, ax = plt.subplots(figsize=(8, 4))

ax.plot(x, np.sin(x), 'b-', lw=2, label='sin')
ax.set_xlabel('x', fontsize=12)
ax.set_ylabel('y', fontsize=12)
ax.set_title('OO API', fontsize=14)
ax.legend(loc='upper right')
ax.grid(True, linestyle='--', alpha=0.5)

fig.tight_layout()
fig.savefig('output.pdf')   # formato vettoriale!
```

Tipi di plot: linee e scatter

ax.plot() · ax.scatter()

plot() — grafici a linee e simboli

Python

```python
x = np.linspace(0, 4*np.pi, 300)

# Forma base
ax.plot(x, np.sin(x))

# Forma stringa colore-stile-marker (MATLAB style)
ax.plot(x, np.sin(x), 'r--')       # rosso, tratteggiato
ax.plot(x, np.cos(x), 'b:o')       # blu, puntini, cerchi

# Forma esplicita (più leggibile)
ax.plot(x, np.sin(x),
        color='steelblue', linestyle='--',
        linewidth=2, marker='o',
        markersize=4, markevery=20,
        alpha=0.8, label='sin(x)')

# Riempimento area
ax.fill_between(x, np.sin(x), 0,
                where=(np.sin(x)>0),
                alpha=0.3, color='steelblue')

# Scale logaritmiche
ax.set_yscale('log')   # 'symlog', 'logit'
```

scatter() — diagramma a dispersione

Python

```python
rng = np.random.default_rng(42)
N   = 150
x_s = rng.standard_normal(N)
y_s = x_s * 0.8 + rng.standard_normal(N) * 0.5
col = rng.uniform(0, 1, N)    # valore per colormap
siz = rng.uniform(20, 200, N) # dimensione marcatore

sc = ax.scatter(x_s, y_s,
                c=col,          # colore da array
                s=siz,          # dimensione variabile
                cmap='viridis',
                alpha=0.7,
                edgecolors='white',
                linewidths=0.5)

cbar = fig.colorbar(sc, ax=ax)
cbar.set_label('Valore', fontsize=10)
```

■ plot vs scatter

plot() è ottimizzato per linee continue (performance su dati grandi).
scatter() supporta colore e dimensione per punto (4D: x, y, colore, size).

Tipi di plot: barre e istogrammi

ax.bar() · ax.barh() · ax.hist()

bar() e barh() — diagrammi a barre

Python

```python
categorie = ['A', 'B', 'C', 'D', 'E']
valori    = [23, 45, 12, 67, 34]
errori    = [2,  4,  1,  5,  3]

# Barre verticali con barre di errore
bars = ax.bar(categorie, valori,
              color='steelblue', edgecolor='white',
              yerr=errori, capsize=4, alpha=0.85)

# Etichette sopra le barre
for bar, v in zip(bars, valori):
    ax.text(bar.get_x() + bar.get_width()/2,
            bar.get_height() + 1,
            str(v), ha='center', fontsize=9)

# Barre orizzontali
ax.barh(categorie, valori, color='coral')

# Grouped bar chart
x_pos = np.arange(len(categorie)); width = 0.35
ax.bar(x_pos - width/2, valori, width, label='2023')
ax.bar(x_pos + width/2, valori[::-1], width, label='2024')
ax.set_xticks(x_pos); ax.set_xticklabels(categorie)
```

hist() — istogramma

Python

```python
rng  = np.random.default_rng(0)
dati = rng.standard_normal(1000)

# Istogramma base
ax.hist(dati, bins=30, color='steelblue',
        edgecolor='white', alpha=0.8)

# Normalizzato a densità di probabilità
ax.hist(dati, bins=30, density=True,
        color='steelblue', alpha=0.6, label='dati')
from scipy.stats import norm
xg = np.linspace(-4, 4, 200)
ax.plot(xg, norm.pdf(xg), 'r-', lw=2, label='N(0,1)')
ax.legend()

# Confronto due distribuzioni
ax.hist(dati,         bins=25, alpha=0.5, label='A')
ax.hist(dati*0.5+1,  bins=25, alpha=0.5, label='B')
ax.legend()

# 2D histogram: heatmap di densità
h = ax.hist2d(x_s, y_s, bins=20, cmap='Blues')
fig.colorbar(h[3], ax=ax)
```

■ Opzioni utili

hist() restituisce (n, bins, patches) per accedere ai bin.

histtype='step' per contorno; stacked=True per barre impilate.

Stili, colori e colormap

Controllo visivo completo dell'aspetto dei grafici

Specificare i colori

Sintassi	Esempio	Descrizione
`Nome HTML`	'steelblue', 'coral'	nomi CSS/HTML
`Hex string`	'#FF5733', '#2E86AB'	RGB esadecimale
`RGB tuple`	(0.1, 0.6, 0.9)	valori 0-1
`RGBA tuple`	(0.1, 0.6, 0.9, 0.5)	con alpha
`Lettera`	'r','g','b','c','m','y'	shortcut base
`CN notation`	'C0','C1','C2',...	ciclo corrente
`Greyscale`	'0.5'	stringa float 0-1
`xkcd`	'xkcd:sky blue'	colori xkcd

Stili di linea e marker

linestyle	Effetto	marker	Simbolo
`'-'  solid`	continua	'o'	cerchio
`'--' dashed`	tratteggiata	's'	quadrato
`':'  dotted`	puntini	'^'	triangolo
`'-.' dashdot`	trattopunto	'*'	stella
`(0,(5,5))`	custom dash	'+'	croce / 'x'

Stili globali (stylesheet)

Python

```python
# Elenco stili disponibili
print(plt.style.available)

# Applicare uno stile globale
plt.style.use('seaborn-v0_8-whitegrid')
plt.style.use('ggplot')

# Parametri globali rcParams
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size']      = 12
```

Colormap

Categoria	Esempi	Uso tipico
`Sequenziale`	viridis, plasma, inferno, cividis	dati ordinati 0→max
`Divergente`	RdBu, coolwarm, bwr, seismic	dati con centro 0
`Qualitativa`	tab10, Set1, Paired, Pastel1	categorie discrete
`Ciclica`	hsv, twilight	angoli, fasi
■ `legacy`	jet, rainbow (da evitare!)	distorcono percezione

■ Scelta della colormap

jet e rainbow distorcono la percezione cromatica.
Preferire viridis, plasma o cividis per dati scientifici.
Per scatter con c=array: ax.scatter(..., cmap='viridis')

Figure e Axes: oggetti e attributi

La gerarchia degli Artist in matplotlib

Gerarchia degli oggetti

Oggetto	Descrizione
`Figure`	contenitore: dimensioni, sfondo, titolo figura
`Axes`	sistema di coordinate con assi, ticks, spine
`Axis`	singolo asse (x o y): scala, limiti, ticks
`Artist`	tutto ciò che viene disegnato: Line2D, Text, Patch…

Attributi principali di Figure

Python – Figure

```
fig, ax = plt.subplots(figsize=(8, 5), dpi=100)

fig.set_size_inches(12, 4)          # ridimensiona
fig.suptitle('Titolo figura', fontsize=16)
fig.tight_layout()                  # evita sovrapposizioni
fig.savefig('out.png', dpi=150, bbox_inches='tight')
fig.savefig('out.pdf')              # formato vettoriale
```

Attributi principali di Axes

```
ax.set_title('Titolo', fontsize=14, pad=10)
ax.set_xlabel('X', fontsize=12); ax.set_ylabel('Y')
ax.set_xlim(0, 10); ax.set_ylim(-1.5, 1.5)
ax.set_aspect('equal')             # rapporto 1:1
ax.set_xticks([0, np.pi, 2*np.pi])
ax.set_xticklabels(['0', 'π', '2π'])
ax.tick_params(axis='both', direction='in', length=6)
ax.grid(True, which='major', ls='--', alpha=0.4)
ax.spines['top'].set_visible(False) # rimuovi bordo
```

Ticks, griglia e spine

Python – ticks & spine

```
ax.minorticks_on()
ax.grid(True, which='minor', ls=':', alpha=0.2)
ax.tick_params(axis='x', labelrotation=45)

# Rimuovere le spine (bordi del grafico)
ax.spines[['top','right']].set_visible(False)
ax.spines['left'].set_linewidth(1.5)
```

Twin axes — scala doppia

Python – twinx()

```
fig, ax1 = plt.subplots(figsize=(8,4))

# Asse Y sinistro
ax1.plot(x, np.sin(x), 'b-', label='sin')
ax1.set_ylabel('sin(x)', color='blue')
ax1.tick_params(axis='y', labelcolor='blue')

# Asse Y destro (condivide asse X)
ax2 = ax1.twinx()
ax2.plot(x, np.exp(x/3), 'r--', label='exp')
```

Inset axes (grafico inserito)

```
axins = inset_axes(ax, width='35%', height='35%',
                   loc='upper right')
axins.plot(x[:50], np.sin(x[:50]), 'r-')
```

Testo, annotazioni, legenda e titoli

Comunicare l'informazione dentro il grafico

Aggiungere testo libero

Python

```python
# Testo in coordinate dati
ax.text(np.pi, 0.5, 'ciao!',
        fontsize=12, color='red',
        ha='center', va='bottom',
        fontweight='bold', rotation=30)

# Testo in coordinate assi (0-1)
ax.text(0.05, 0.95, 'Nota: dati simulati',
        transform=ax.transAxes,
        fontsize=9, color='grey',
        verticalalignment='top')

# Annotazione con freccia
ax.annotate('massimo',
            xy=(np.pi/2, 1),    # punta freccia
            xytext=(2, 0.7),    # posizione testo
            fontsize=11,
            arrowprops=dict(arrowstyle='->', lw=1.5))

# LaTeX nel testo
ax.set_xlabel(r'$\theta$ (rad)', fontsize=13)
ax.set_title(r'$f(\theta)=\sin(\theta)$', fontsize=14)
```

Legenda

Python

```python
# Legenda automatica — usa label=
ax.plot(x, np.sin(x), label='sin(x)')
ax.plot(x, np.cos(x), label='cos(x)')
ax.legend()

# Posizione e stile
ax.legend(loc='upper right',
          fontsize=11,
          title='Funzioni',
          framealpha=0.9,
          ncols=2)

# Legenda fuori dal grafico
ax.legend(bbox_to_anchor=(1.02, 1),
          loc='upper left',
          borderaxespad=0.)
fig.tight_layout()  # necessario!

# Handles manuali
from matplotlib.lines import Line2D
handles = [
    Line2D([0],[0], color='b', lw=2, label='Misure'),
    Line2D([0],[0], color='r', ls='--', label='Modello'),
]
ax.legend(handles=handles)
```

■ Posizionamento legenda

loc='best' sceglie automaticamente la posizione meno sovrapposta.
Per molte curve: ncols=2 o spostare fuori con bbox_to_anchor.

Grafici multipli nella stessa figura

subplots · GridSpec · subplot_mosaic

plt.subplots() — griglia regolare

Python – subplots

```
# 2 righe × 3 colonne
fig, axes = plt.subplots(nrows=2, ncols=3,
                         figsize=(12, 7),
                         sharex=True,
                         constrained_layout=True)

# axes è array numpy shape (2, 3)
axes[0, 0].plot(x, np.sin(x))
axes[0, 1].scatter(x_s, y_s, s=10)
axes[1, 2].hist(dati, bins=20)

# Iterazione su tutti gli assi
for ax in axes.flat:
    ax.grid(True, alpha=0.3)
axes[1, 1].set_visible(False)  # nasconde asse vuoto
```

add_subplot() — inserimento manuale

```
fig = plt.figure(figsize=(10, 6))
ax1 = fig.add_subplot(2, 2, 1)   # riga 2, col 2, pos. 1
ax2 = fig.add_subplot(2, 2, 2)
ax3 = fig.add_subplot(2, 1, 2)   # occupa tutta la riga 2
```

GridSpec — layout avanzato

Python – GridSpec

```
import matplotlib.gridspec as gridspec

fig = plt.figure(figsize=(12, 8))
gs  = gridspec.GridSpec(3, 3, hspace=0.4, wspace=0.4)

ax1 = fig.add_subplot(gs[0, :])   # tutta la riga 0
ax2 = fig.add_subplot(gs[1:, 0])  # 2 righe, col 0
ax3 = fig.add_subplot(gs[1, 1])
ax4 = fig.add_subplot(gs[2, 2])
```

subplot_mosaic() — layout descrittivo (Python 3.8+)

```
mosaic = [
    ['grande', 'grande', 'destra'],
    ['basso_sx', 'basso_cx', 'destra'],
]
fig, axd = plt.subplot_mosaic(
    mosaic, figsize=(10, 6),
    constrained_layout=True)

# Accesso tramite chiave stringa
axd['grande'].set_title('Pannello principale')
axd['destra'].set_facecolor('#EEF')
```

■ Layout e spaziatura

constrained_layout=True evita sovrapposizioni tra titoli e assi.
subplot_mosaic è la soluzione più leggibile per layout asimmetrici.

Grafici 3D

mpl_toolkits.mplot3d — superfici, linee e scatter in 3 dimensioni

Setup

Python – setup

```
from mpl_toolkits.mplot3d import Axes3D
import numpy as np, matplotlib.pyplot as plt

# Creare un asse 3D
fig, ax = plt.subplots(
    subplot_kw={'projection': '3d'}, figsize=(7,6))
ax.set_xlabel('X'); ax.set_ylabel('Y'); ax.set_zlabel('Z')
```

Scatter 3D

```
rng = np.random.default_rng(0); N = 200
xs = rng.standard_normal(N)
ys = rng.standard_normal(N)
zs = xs**2 + ys**2 + rng.normal(0, 0.3, N)

sc = ax.scatter(xs, ys, zs,
                c=zs, cmap='viridis', s=30, alpha=0.7)
fig.colorbar(sc, ax=ax, shrink=0.5)
```

Linea 3D (curva parametrica)

```
t = np.linspace(0, 4*np.pi, 300)
ax.plot3D(np.cos(t), np.sin(t),
          t/(4*np.pi), 'purple', lw=2)
ax.view_init(elev=30, azim=45)  # punto di vista
```

Superficie 3D (surface plot)

Python – surface

```
x1 = np.linspace(-3, 3, 80)
y1 = np.linspace(-3, 3, 80)
X, Y = np.meshgrid(x1, y1)
Z    = np.sin(np.sqrt(X**2 + Y**2))

surf = ax.plot_surface(X, Y, Z,
                       cmap='coolwarm', alpha=0.9)
fig.colorbar(surf, ax=ax, shrink=0.5)
```

Wireframe e contour proiettato

```
# Wireframe (griglia senza riempimento)
ax.plot_wireframe(X, Y, Z,
                  rstride=5, cstride=5, alpha=0.4)

# Contour proiettato sul piano Z
ax.contourf(X, Y, Z, zdir='z',
            offset=Z.min(), cmap='coolwarm', alpha=0.5)
```

Controllo del punto di vista e aspetto

```
ax.view_init(elev=30, azim=45)   # elevazione e azimuth
ax.set_box_aspect([1, 1, 0.5])   # comprimi asse Z
ax.xaxis.pane.fill = False       # togli pannelli sfondo
```

■ Alternative avanzate

mplot3d è statico (CPU). Per 3D interattivo e ad alte prestazioni:
usare plotly, vispy o mayavi.

Riepilogo e Risorse

matplotlib in una pagina

Argomenti trattati

- **Importazione:** import matplotlib.pyplot as plt | %matplotlib inline
- **API pyplot vs OO:** plt.plot() rapido vs fig, ax = plt.subplots() consigliato
- **plot():** linee, marker, fill_between, scale log, color/ls/lw/marker
- **scatter():** punti con c= (colormap) e s= (dimensione variabile)
- **bar() / barh():** verticali/orizzontali, grouped, barre di errore, etichette
- **hist():** bins, density, overlay, hist2d
- **Colori:** nomi, hex, RGB, CN, xkcd — linestyle: -, --, :, -.
- **Colormap:** viridis/plasma (seq.) · RdBu (div.) · tab10 (qual.)
- **Stili globali:** plt.style.use() · plt.rcParams[] · context manager with
- **Figure & Axes:** figsize, dpi, suptitle, xlim/ylim, ticks, spine, grid
- **Twin axes:** ax.twinx() / twiny() per scala doppia sullo stesso grafico
- **Testo:** ax.text() · ax.annotate() con freccia · LaTeX · transAxes
- **Legenda:** label=, legend(loc, ncols, bbox_to_anchor, handles)
- **Subplots:** subplots(nrows,ncols) · GridSpec · subplot_mosaic
- **3D:** projection='3d', plot_surface, plot3D, scatter, view_init

Risorse ufficiali

- **Documentazione ufficiale:** https://matplotlib.org/stable/
- **Tutorials:** https://matplotlib.org/stable/tutorials/
- **Esempi e gallery:** https://matplotlib.org/stable/gallery/
- **API reference (pyplot):** https://matplotlib.org/stable/api/pyplot_summary.html
- **API reference (Axes):** https://matplotlib.org/stable/api/axes_api.html
- **Cheat sheets:** https://matplotlib.org/cheatsheets/
- **mplot3d toolkit:** https://matplotlib.org/stable/api/toolkits/mplot3d.html
- **Colormap reference:** https://matplotlib.org/stable/gallery/color/colormap_reference.html

Ecosistema e librerie complementari

`seaborn` — statistiche — API di alto livello su matplotlib
`plotly` — grafici interattivi HTML, 3D avanzato
`bokeh` — visualizzazioni web interattive
`altair` — grammatica della visualizzazione (Vega-Lite)
`pandas .plot()` — grafici diretti da DataFrame
`scikit-learn` — ConfusionMatrixDisplay, RocCurveDisplay, ecc.